

Universidad Autónoma de Madrid
Departamento de Informática

Sistemas Informáticos II

Práctica 3 — Memoria

Medición de rendimiento con Apache JMeter

[NOMBRE ALUMNO 1]
[NOMBRE ALUMNO 2]

Curso 2025-2026

Ejercicio 1: Plan de pruebas para P1-base

Se ha definido el plan de pruebas completo para el proyecto P1-base siguiendo las instrucciones del enunciado. El plan se encuentra en el fichero **P3_P1-base.jmx** y contiene los siguientes elementos:

Estructura del plan: Un Thread Group "P1-base" que contiene un contador, un CSV Data Set Config para leer los datos de tarjetas desde *data.csv*, tres HTTP Requests (GET tarjeta, POST tarjeta con extracción de CSRF token, y POST pago), dos Gaussian Random Timers (offset=3000ms, σ =500ms) para simular tiempos de pensamiento, un Response Assertion que verifica "Pago Registrado", un HTTP Cookie Manager a nivel de Test Plan, y los listeners Aggregate Report, View Results Tree y Summary Report.

Las variables locales del Thread Group son: *entrypoint=visaApp*, *port=18000*, *host=localhost*. El número de threads se parametriza mediante $\$_{P(users,10)}$ y el Loop Count con la variable global *samples*.

Captura del Aggregate Report

[PENDIENTE] Insertar aquí la captura de pantalla del Aggregate Report tras ejecutar el plan con 10 usuarios y samples=10.

Significado de las columnas del Aggregate Report

Columna	Significado
Label	Nombre del HTTP Request analizado (Get Tarjeta Form, POST Tarjeta, Post Pago).
# Samples	Número total de solicitudes enviadas para ese sampler.
Average	Tiempo de respuesta promedio en milisegundos.
Median	Mediana del tiempo de respuesta (percentil 50). Valor central de la distribución.
90% Line	Percentil 90 del tiempo de respuesta. El 90% de las peticiones tardaron menos que este valor.
95% Line	Percentil 95 del tiempo de respuesta.
99% Line	Percentil 99 del tiempo de respuesta.
Min	Tiempo mínimo de respuesta registrado (ms).
Max	Tiempo máximo de respuesta registrado (ms).
Std. Dev.	Desviación estándar del tiempo de respuesta. Indica la variabilidad.
Error %	Porcentaje de solicitudes que fallaron (no superaron la aserción o dieron error HTTP).
Throughput	Solicitudes procesadas por segundo (req/s). Métrica clave de rendimiento.
Received KB/sec	Kilobytes recibidos por segundo desde el servidor.
Sent KB/sec	Kilobytes enviados por segundo al servidor.
Avg. Bytes	Tamaño promedio de la respuesta en bytes.

Ejercicio 2: Generalización a tres proyectos

Se ha generalizado el plan de pruebas para que funcione con los tres proyectos: P1-base, P1-ws-frontend/backend y P2-rcp-frontend/backend. El plan se encuentra en el fichero **P3-projects.jmx**.

La generalización se ha realizado duplicando el Thread Group original y modificando las variables locales (*entrypoint*, *host* y *port*) de cada copia para que apunten a la aplicación web correspondiente. Cada Thread Group contiene su propia lógica completa (contador, CSV Data Set Config, HTTP Requests con extracción de CSRF, timers y assertions), compartiendo las variables globales *samples* y *debug* a nivel de Test Plan.

Se ha activado la opción *Run Thread Groups consecutively* en el Test Plan para que los grupos de hilos se ejecuten secuencialmente, permitiendo medir el rendimiento de cada proyecto de forma independiente.

Thread Group	entrypoint	port	host
P1-base	visaApp	18000	vm2 (o localhost)
P1-ws	visaAppWSFrontend	18003	vm3
P2-rcp	visaAppRPCFrontend	18005	vm3

Cuestión 1: Diferencias entre Thread Groups P1-base y P1-ws

Las principales diferencias entre el Thread Group que prueba P1-base y el que prueba P1-ws son las siguientes:

- 1. Variable *entrypoint*:** En P1-base el *entrypoint* es *visaApp*, correspondiente a la aplicación monolítica. En P1-ws es *visaAppWSFrontend*, que corresponde al frontend del proyecto basado en web services. Esto cambia las rutas de las peticiones HTTP (por ejemplo, */visaApp/tarjeta/* vs */visaAppWSFrontend/tarjeta/*).
- 2. Variable *host*:** En P1-base las peticiones se dirigen a *vm2* (donde corre el servidor monolítico con gunicorn). En P1-ws las peticiones van a *vm3* (donde corre el frontend del web service), que a su vez se comunica internamente con el backend en *vm2*.
- 3. Variable *port*:** Cada proyecto escucha en un puerto diferente. P1-base utiliza el puerto 18000 mientras que P1-ws utiliza el puerto 18003.
- 4. Arquitectura subyacente:** Aunque desde el punto de vista de JMeter las peticiones son similares (GET tarjeta, POST tarjeta, POST pago), en P1-base todo se procesa en un único servidor, mientras que en P1-ws el frontend recibe la petición y delega la lógica de negocio al backend mediante llamadas REST API. Esto introduce latencia adicional por la comunicación entre frontend y backend.

Ejercicio 3: Test de estrés con 1000 threads

Se ha ejecutado el plan de pruebas con el Thread Group "P1-base" configurado con 1000 threads y un Ramp-Up de 1 segundo. Esto intenta crear 1000 conexiones concurrentes en solo 1 segundo. Se presentan las tres hipótesis planteadas:

- (a) JMeter no es capaz de obtener los recursos para crear 1000 peticiones en 1 segundo.
- (b) JMeter crea las 1000 peticiones pero el throughput real es inferior; todas las peticiones son finalmente atendidas.
- (c) La capacidad del servidor se desborda y las peticiones se pierden o devuelven errores.

Resultados experimentales

[PENDIENTE] Insertar aquí los resultados de la ejecución con 1000 threads: captura del resumen de JMeter (throughput, errores, tiempos).

Monitorización con nmon

[PENDIENTE] Insertar capturas de nmon de vm1 (base de datos) y vm2 (servidor): uso de CPU, memoria, disco y red durante la prueba.

Análisis

[PENDIENTE] Analizar cuál de las hipótesis (a), (b) o (c) se cumple, basándose en los datos recogidos. Argumentar con los datos de throughput, tasa de errores y uso de recursos observados en nmon.

Ejercicio 4: ¿Cuándo comienza el segundo loop?

En un Thread Group con Loop Count > 1, cada thread ejecuta la secuencia de peticiones (GET tarjeta → POST tarjeta → POST pago) múltiples veces. La pregunta es: ¿cuándo comienza el segundo bucle?

- (a) Tan pronto como todas las peticiones del primer bucle se han *creado* (enviado).
- (b) Tan pronto como todas las peticiones del primer bucle han sido *atendidas* (se ha recibido respuesta).

Respuesta

La respuesta correcta es la **opción (b)**: el segundo bucle comienza cuando todas las peticiones del primer bucle han sido atendidas (se han recibido sus respuestas).

Esto se debe a que en JMeter cada thread ejecuta su flujo de forma secuencial. Un thread individual envía la primera petición, espera la respuesta, ejecuta el timer de espera, envía la segunda petición, espera la respuesta, y así sucesivamente. Solo cuando ha completado todas las peticiones del loop actual (incluyendo recibir todas las respuestas) comienza la siguiente iteración del bucle.

Es importante distinguir que la concurrencia en JMeter se logra mediante *múltiples threads*, no mediante ejecución asíncrona dentro de un mismo thread. Cada thread es independiente y ejecuta su secuencia de forma síncrona.

Evidencia experimental

[PENDIENTE] Insertar captura del View Results Tree mostrando los timestamps de las peticiones de un mismo thread en el primer y segundo loop. Verificar que el segundo loop comienza después de recibir la respuesta de la última petición del primero.

Ejercicio 5: Curvas de productividad de los tres proyectos

Se ha calculado la curva de productividad (throughput vs número de usuarios) para cada uno de los tres proyectos. Cada experimento se ha repetido 10 veces (samples=10) y se ha ejecutado en modo CLI para evitar la sobrecarga de la interfaz gráfica. El ramp-up se ha configurado suficientemente pequeño respecto al tiempo total de ejecución.

Los grupos de hilos se ejecutan secuencialmente (no concurrentemente) para medir el rendimiento de cada proyecto de forma independiente.

Datos de throughput

[PENDIENTE] Insertar tabla con los datos de throughput (req/s) para cada número de usuarios probado (1, 2, 3, 5, 7, 10, 15, 20, 25, 30) × 3 proyectos.

Gráfica: P1-base

[PENDIENTE] Insertar gráfica de curva de productividad de P1-base.

Gráfica: P1-ws

[PENDIENTE] Insertar gráfica de curva de productividad de P1-ws.

Gráfica: P2-rpc

[PENDIENTE] Insertar gráfica de curva de productividad de P2-rpc.

Gráfica comparativa

[PENDIENTE] Insertar gráfica comparativa de los tres proyectos.

Análisis comparativo

[PENDIENTE] Analizar qué implementación es más eficiente y por qué. Se espera que P1-base (monolítico) tenga mayor throughput que P1-ws y P2-rpc, ya que estas últimas introducen latencia adicional por la comunicación frontend-backend (REST API o RPC). Argumentar con los datos obtenidos.

Ejercicio 6: gunicorn vs servidor de desarrollo

Se ha comparado el rendimiento de **gunicorn** (servidor de producción) frente al servidor de desarrollo de Django (*python manage.py runserver*) para el proyecto P1-base. Para el servidor de desarrollo se ha utilizado *python manage.py runserver 0.0.0.0:8000* para permitir conexiones externas.

Datos de throughput

[PENDIENTE] Insertar tabla comparativa de throughput: gunicorn vs runserver para cada número de usuarios.

Gráfica comparativa

[PENDIENTE] Insertar gráfica comparativa de curvas de productividad: gunicorn (azul) vs runserver (rojo).

Análisis

[PENDIENTE] Analizar las diferencias. Se espera que gunicorn ofrezca mayor throughput ya que está diseñado para producción con gestión eficiente de workers y conexiones. El servidor de desarrollo es single-threaded y no está optimizado para manejar carga concurrente. Argumentar con datos.

Ejercicio 7: 2 workers + 2 CPUs + SESSION_ENGINE=db

Se ha configurado la máquina virtual vm2 con **2 CPUs** y se ha lanzado gunicorn con **2 workers**. Adicionalmente, se ha modificado el `SESSION_ENGINE` en `settings.py` a `django.contrib.sessions.backends.db` para almacenar las sesiones en la base de datos en lugar de en caché de memoria.

El cambio de `SESSION_ENGINE` es necesario porque con más de un worker, cada worker tiene su propio espacio de memoria. Si las sesiones se almacenan en caché local (`LocMemCache`), un worker no puede acceder a las sesiones creadas por otro worker. Al usar la base de datos como backend de sesiones, todos los workers comparten el mismo almacenamiento de sesiones.

Configuración

```
# settings.py
SESSION_ENGINE = 'django.contrib.sessions.backends.db'

# Comando gunicorn
gunicorn visaSite.wsgi:application --bind 0.0.0.0:8000 --workers 2
```

Gráfica comparativa

[PENDIENTE] Insertar gráfica comparativa: 1 worker/1 CPU vs 2 workers/2 CPUs.

Análisis

[PENDIENTE] Analizar el impacto de duplicar workers y CPUs. Se espera una mejora significativa del throughput ya que ahora hay dos procesos worker atendiendo peticiones en paralelo, cada uno en su propia CPU. Sin embargo, el uso de `SESSION_ENGINE=db` introduce latencia adicional por las consultas a la BD para cada operación de sesión.

Discutir el trade-off. Argumentar con datos.

Cuestión 2: ¿Dónde cambiar `SESSION_ENGINE` en `ws-frontend/backend`?

Si repetimos el ejercicio 7 usando el proyecto `P1-ws-frontend/backend` en lugar de `P1-base`, ¿en qué fichero `settings.py` tenemos que modificar la variable `SESSION_ENGINE`?

- (a) En el del backend
- (b) En el del frontend
- (c) En ambos
- (d) En ninguno

Respuesta

La respuesta correcta es la **opción (b): en el del frontend**.

En la arquitectura `ws-frontend/backend`, el **frontend** es el componente que recibe las peticiones HTTP directamente del usuario (a través de JMeter). Es el frontend quien gestiona la sesión HTTP del usuario: cuando el usuario envía el formulario de tarjeta, el frontend almacena el *numeroTarjeta* en una variable de sesión para recuperarlo después en el formulario de pago.

El backend, por su parte, expone una API REST y no mantiene estado de sesión HTTP con el usuario final. El backend recibe llamadas del frontend con los datos ya procesados y simplemente interactúa con la base de datos.

Por tanto, es en el `settings.py` del frontend donde se debe cambiar `SESSION_ENGINE` a `django.contrib.sessions.backends.db`, ya que es el único componente que utiliza sesiones HTTP. Si el frontend se ejecuta con múltiples workers de gunicorn, necesita que las sesiones sean accesibles por todos los workers a través de la base de datos.